

PRACTICE: From Experience to Expertise in Self-Evolving Embodied Agents

Ziyi Bai^{1*}, Siqi Li^{2,3*}, Tinglei Huang², Börje F. Karlsson^{1†}

¹Beijing Academy of Artificial Intelligence (BAAI)

²Institute of Software, Chinese Academy of Sciences

³University of Chinese Academy of Sciences

Abstract

Recent studies have shown that multimodal large language models (MLLMs) can serve as embodied agents, translating language instructions and visual observations into executable plans. However, building agents that can continually improve through interaction and rapidly adapt to their environments remains challenging. Summing up experience from past interaction trajectories provides a promising solution, but existing experience-based methods often rely on manually designed prompting workflows to extract and update skills. Such fixed procedures may struggle to learn updated skills from new and diverse experiences. We introduce PRACTICE, which trains a skill learner to discover and maintain a persistent skill library from past interaction trajectories while keeping the task executor frozen. Given the historical accumulated skills and incoming trajectories, the skill learner produces structured batch-edits that add, refine, merge, or remove skills, and then hierarchical consolidate all collected edits into a consistent updated skill library. We train the learner with a two-stage curriculum. First, it learns basic skill generation and library maintenance from oracle trajectories. Then, by contrasting successful and failed trajectories from heterogeneous executors on the same tasks, it learn to identify invalid action patterns and recovery strategies. Finally, we apply online skill-edit distillation to align the skill learner with a stronger teacher on its current edit distribution to further improves the policy. Experiments demonstrate that a compact skill learner delivers consistent performance improvements across successive library-update rounds for multiple frozen executors. On EB-ALFRED and EB-Habitat, PRACTICE further outperforms the strongest experience-based baselines by 9.7 and 2.6 percentage points respectively. Project resources are publicly available at: <https://baai-agents.github.io/PRACTICE>.

1 Introduction

Embodied task planning has emerged as an important testbed for general-purpose agents, requiring them to interpret human instructions, decompose complex goals into executable steps, and accomplish those goals through interaction with the environment (Yang et al. 2025; Liu et al. 2025; Xu, Zheng, and Mu 2026). Recent MLLMs have demonstrated strong instruction understanding, visual perception, and planning capabilities in such environments (Ahn et al. 2022; Yao

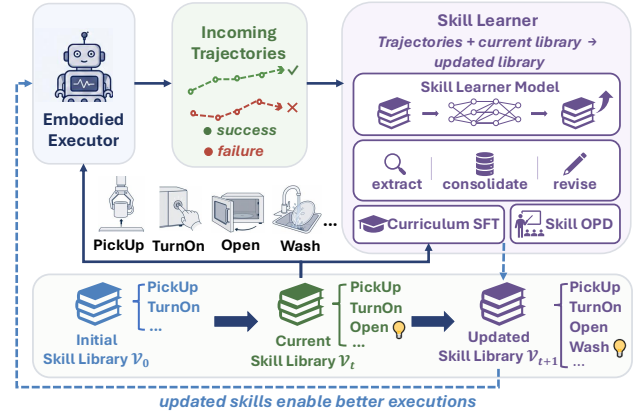


Figure 1: **Overview of our skill learning framework.** A trainable skill learner extracts, consolidates, and revises reusable skills from successful and failed execution trajectories, progressively updating an external skill library. The updated skills guide the embodied executor toward improved task execution.

et al. 2023; Song et al. 2023; Yuan et al. 2025). To further improve their reliability, a growing body of work directly post-trains the executor using expert or model-generated trajectories, environment-grounded supervision, and task-level interaction rewards, typically through supervised fine-tuning, imitation learning, or reinforcement learning (Zhai et al. 2024; Wang et al. 2025; Chen et al. 2025; Wu et al. 2026; Xu, Zheng, and Mu 2026). These methods enable the model to internalize task- and environment-specific experience in its parameters. Nevertheless, challenging instructions and long-horizon interactions continue to expose failures. In such cases, effective adaptation requires more than generating a plan: the agent must reflect on its execution outcomes and preserve useful experience for future decisions.

Therefore, a line of experience-augmented methods externalizes such experience as persistent memories or reusable skills. Memory-based methods extract and retrieve episodic or semantic records (Zhang et al. 2025; Chhikara et al. 2025; Zhang, Fu, and Yan 2026), whereas skill-based methods distill trajectories into action procedures, lessons, or executable rules (Ni et al. 2026; Jiang et al. 2026; Yang

*These authors contributed equally.

†Corresponding author.

et al. 2026; Ju et al. 2026; Ding et al. 2026). However, most of the current experience-augmented works rely on fixed, prompt-engineered workflows to generate, refine, merge, or remove skill entries. Consequently, although the library content evolves, its update policy does not learn from accumulated experience or downstream execution outcomes, limiting its adaptability across tasks and environments. We therefore ask: *Can an embodied agent learn not only new skills, but also how to grow and refine those skills?*

To address this, we propose **PRACTICE**, which treats the skill library as continuously evolving procedural knowledge and the skill learner as a learnable update policy. As shown in the Figure 1, the learner and the library are improved iteratively. The current library first guides task execution and shapes the collected trajectories, which are then used to improve the learner’s skill-refinement policy. The updated learner, in turn, extracts reusable experience from these trajectories and produces refined skills. The updated library guides the next round of execution and provides new experience for further learning. In this way, the policy for refining skills and the procedural knowledge being refined improve together.

We instantiate this progression through three training stages. Stage 0 uses successful oracle trajectories to cold-start skill generation and library maintenance. Stage 1 contrasts successful and failed trajectories from heterogeneous executors on the same tasks, teaching the learner to identify invalid condition for skill application and derive strategies for failure recovery. Finally, Stage 2 introduces online skill-refinement policy distillation (SKILL OPD), where the learner generates edits from incoming trajectories and aligns its behavior with a stronger teacher on its own edit distribution. Each stage inherits the learner and skill library from the preceding stage, progressively improving both the update policy and the procedural knowledge available to the executor.

We evaluate **PRACTICE** on two high-level embodied task-planning benchmarks, EB-ALFRED and EB-Habitat, from EmbodiedBench (Yang et al. 2025). Across heterogeneous executors, **PRACTICE** consistently improves task performance and outperforms the strongest experience-augmented baselines by 9.7 and 2.6 percentage points, respectively. These findings suggest that sustained agent improvement depends not merely on evolving what an agent knows, but on learning how its procedural knowledge should evolve through experience. Our main contributions are:

- We formulate skill evolution as a coupled learning problem and introduce a new framework that treats the skill library as evolving procedural knowledge and the skill learner as a learnable update policy.
- We propose a two-stage training framework combining curriculum supervised fine-tuning with online skill-edit distillation, enabling failure-aware skill maintenance and iterative refinement.
- We outperform the strongest experience-based baselines on EB-ALFRED and EB-Habitat while consistently improving multiple frozen executors.

2 Related Works

2.1 Embodied Task Planning

Embodied task planning requires agents to translate language instructions and visual observations into coherent action sequences. Recent work primarily improves this capability by post-training MLLM/VLM executors. RL4VLM applies reinforcement learning to VLM decision policies (Zhai et al. 2024), while REBP and RoboGPT-R1 combine supervised initialization with reinforcement learning for structured reasoning, task completion, and action consistency (Wu et al. 2025; Liu et al. 2025). VAGEN reinforces state estimation and transition modeling under partial observability (Wang et al. 2025). More recent methods exploit richer task supervision: ERA combines embodied priors with on-line RL (Chen et al. 2025), ORBIT performs on-policy fine-tuning with offline rewards (Wu et al. 2026), and RoboAgent trains coordinated vision-language capabilities through behavior cloning, trajectory aggregation, and reinforcement learning (Xu, Zheng, and Mu 2026). These approaches improve planning by internalizing task knowledge and interaction policies into the executor’s parameters.

2.2 Experience-Augmented Agents

Because executor post-training typically requires task-specific trajectories, another line of work externalizes experience as reusable skills or memories. Prompt-based methods summarize trajectories and maintain external knowledge through predefined operations: Trace2Skill consolidates trajectory-level lessons (Ni et al. 2026), AutoSkill maintains skill artifacts through addition, merging, and removal (Yang et al. 2026). Memory-based methods organize experience into different memory structures (Zhang et al. 2025; Zhang, Fu, and Yan 2026; Ding et al. 2026), while ELITE and EmbodiSkill use reflective updates to refine reusable strategies or procedural skills (Wei et al. 2026; Ju et al. 2026). However, their maintenance logic is primarily prescribed through prompts and heuristics. Therefore, researches like MemCtrl trains a memory gate to retain or discard observations (Dorbala and Manocha 2026), SkillOS learns a curator for an external skill repository (Ouyang et al. 2026). In contrast, **PRACTICE** progressively trains a dedicated learner to grow and refine a persistent skill library.

3 Methodology

3.1 Problem Formulation

Our agent system consists of an embodied task executor, a persistent skill library, and a learnable skill learner: the library guides task execution, while the learner converts the resulting trajectories into library updates, forming a closed loop of skill-level self-evolution.

Embodied task executor. We consider an embodied task executor that interacts with a physically grounded environment $\mathcal{E}$ under natural-language instructions. Each task is modeled as a partially observable decision process. Given an instruction I , the executor receives an observation o_t at each time step t and selects an action a_t from the action space

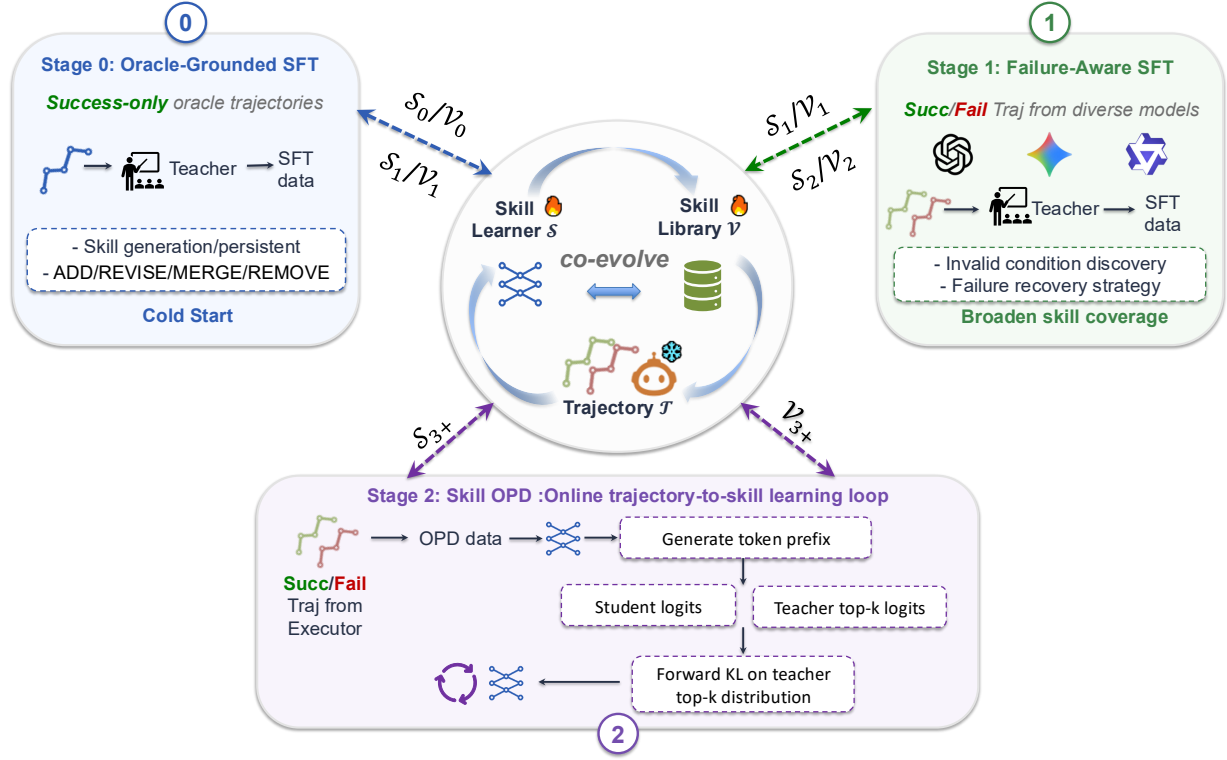


Figure 2: **Training pipeline of PRACTICE.** Stages 0–1 use oracle-grounded and failure-aware SFT to initialize skill learning while Stage 2 applies SKILL OPD to match a teacher’s token distributions on student-generated prefixes via forward KL.

$\mathcal{A}$. We denote the executor policy by π_{exec} . An execution trajectory is represented as

$$\tau = (I, o_1, a_1, \dots, o_T, a_T, y), \quad (1)$$

where T is the trajectory length and $y \in \{0, 1\}$ denotes the final task-success outcome. Throughout our framework, the parameters of π_{exec} remain frozen.

Skill Library. The skill library $\mathcal{V}$ stores reusable procedural knowledge available to the executor for embodied task execution. The skill library is a collection of skill cards:

$$\mathcal{V} = \{v_j\}_{j=1}^{|\mathcal{V}|}. \quad (2)$$

We define a skill card as a parameterized procedural unit. Each skill card v_j consists of an action pattern sequence p_j and a structured usage specification u_j :

$$v_j = (p_j, u_j), \quad p_j = [(a_1, \phi_1), \dots, (a_L, \phi_L)], \quad (3)$$

where $a_l \in \mathcal{A}$ is a primitive action and ϕ_l denotes its arguments, such as an object category or a target state. For example, the pattern (pick_up, object_target) could describe a reusable “picking-up” skill. The usage specification u_j describes the semantic scope and execution constraints of the skill, such as *when* the skill is applicable and *how* the executor should recover from failure.

Skill-guided execution. The task executor is augmented with the current skill library $\mathcal{V}$. The library provides procedural guidance for action planning, including applicable

action patterns, execution preconditions, expected effects, and recovery strategies. Let h_t denotes the interaction history available at step t . The executor selects its next action according to

$$a_t \sim \pi_{\text{exec}}(\cdot | I, h_t, \mathcal{V}). \quad (4)$$

Although the parameters of π_{exec} remain unchanged, different versions of the skill library induce different action distributions and therefore different trajectory distributions.

Skill learner. We introduce the skill learner $\mathcal{S}$ as a parameterized library-update policy. It interprets execution trajectories, extract reusable procedural experience, and determine how the persistent skill library should grow and be refined over time. At update round i , the current library $\mathcal{V}_i$ is provided to the task executor to collect a new trajectory set $\mathcal{T}_i$. Then the skill learner predicts a set of structured edits to grow the skill library like:

$$\Delta = \mathcal{S}(\mathcal{V}, \mathcal{T}). \quad (5)$$

Here, Δ may contain operations that ADD, REVISE, MERGE, or REMOVE skill cards. The predicted edits are applied to produce the next skill library:

$$\mathcal{V} = \text{Apply}(\mathcal{V}, \Delta). \quad (6)$$

The skill learner and skill library thus form a coupled evolution process. An improved skill learner produces more reliable library edits, while the updated library changes the executor’s future trajectory distribution and provides new on-policy evidence for learning subsequent update policies.

3.2 PRACTICE Overview

As illustrated in Figure 2, instead of relying on a fixed, prompt-engineered procedure for constructing and maintaining a skill library, PRACTICE establishes a coupled update cycle between a learnable skill learner and a persistent skill library.

(1) Learner update ($\mathcal{S}_i \xrightarrow{(\mathcal{V}_i, \mathcal{T}_i)} \mathcal{S}_{i+1}$). At round i , the current library $\mathcal{V}_i$ guides the executor and induces trajectories $\mathcal{T}_i$, which provide supervision for improving the skill learner.

(2) Library update ($\mathcal{V}_i \xrightarrow{\mathcal{S}_{i+1}} \mathcal{V}_{i+1}$). The updated learner interprets $\mathcal{T}_i$ in the context of $\mathcal{V}_i$ and produces structured edits to construct $\mathcal{V}_{i+1}$.

The updated library subsequently changes the executor’s trajectory distribution and provides new interaction evidence for the next round:

$$\mathcal{V}_i \rightarrow \mathcal{T}_i \rightarrow \mathcal{S}_{i+1} \rightarrow \mathcal{V}_{i+1} \rightarrow \mathcal{T}_{i+1}. \quad (7)$$

We instantiate this coupled progression through three stages: oracle-grounded SFT, failure-aware SFT, and online skill-refinement policy distillation (SKILL OPD). Each stage inherits the learner and library from the preceding stage, progressively improving both the skill-refinement policy and the procedural knowledge. We introduce each stage below.

Initial Skill Library Construction We construct the initial library through action-pattern discovery inspired by byte-pair encoding (BPE) (Sennrich, Haddow, and Birch 2016). We first collect successful trajectories from the executor without skill augmentation and convert them into sequences of primitive actions. Frequent adjacent action subsequences are then iteratively merged to discover recurring multi-step patterns. Finally, the initial skill learner $\mathcal{S}_0$, initialized from a pretrained base model, consolidates these patterns into structured skill cards, forming the initial library $\mathcal{V}_0$.

3.3 Curriculum Training of the Skill Learner

Following curriculum learning, which organizes training examples in a meaningful progression from simpler to more difficult concepts (Bengio et al. 2009), we cold-start the skill learner through two stages. Stage 0 uses successful oracle trajectories to establish fundamental skill generation, editing, and consolidation capabilities. Stage 1 subsequently introduces heterogeneous executor rollouts containing both successes and failures, enabling the learner to perform failure-aware skill attribution and revision.

Structured skill-editing supervision. At stage i , the learner receives the current library $\mathcal{V}_i$ and a trajectory batch $B_i \subseteq \mathcal{T}_i$. A strong teacher model generates a structured target edit set:

$$\Delta_i^* = \mathcal{G}(\mathcal{V}_i, B_i), \quad (8)$$

where $\mathcal{G}$ denotes the teacher-based data generation process. Depending on the trajectory evidence and current library state, Δ_i^* may contain ADD, REVISE, MERGE, and REMOVE operations. Given the resulting curriculum dataset $\mathcal{D}_k$, the

skill learner is optimized with the autoregressive SFT objective:

$$\mathcal{L}_{\text{SFT}}^{(i)} = -\mathbb{E}_{(x,y) \sim \mathcal{D}_i} \left[\sum_{t=1}^{|y|} \log p_{\mathcal{S}_{i+1}}(y_t \mid x, y_{<t}) \right], \quad (9)$$

where x contains the current library and trajectory evidence, and y denotes the teacher-generated editing output.

Stage 0: Oracle-Grounded Skill Editing Stage 0 uses only successful oracle trajectories. This controlled setting allows the learner to acquire basic skill-editing capabilities before it must reason about noisy and potentially ambiguous failures.

To simulate the streaming arrival of executor experience, we randomly sample N oracle trajectories to form each trajectory batch:

$$B_{0,b} \subseteq \mathcal{T}^{\text{oracle}}, \quad |B_{0,b}| = N, \quad (10)$$

where b denotes the batch index. We construct three complementary types of Stage-0 supervision.

Skill generation from an empty library. The first type removes all existing skill context and asks the teacher to construct reusable skill cards directly from a trajectory batch:

$$\Delta_{0,b}^{\text{gen}} = \mathcal{G}(\emptyset, B_{0,b}). \quad (11)$$

These examples teach the learner to identify reusable procedural abstractions without relying on pre-existing skill cards.

Editing the initialized library. The second type provides the BPE-initialized library $\mathcal{V}_0$ together with the same trajectory evidence:

$$\Delta_{0,b}^{\text{edit}} = \mathcal{G}(\mathcal{V}_0, B_{0,b}). \quad (12)$$

The teacher determines whether the evidence supports adding a missing skill, revising an incomplete skill, merging overlapping cards, removing an unsupported card, or preserving the existing library. These examples train the learner to make localized edits rather than regenerate the whole library.

Cross-batch library consolidation. Edits obtained from individual batches may describe the same procedure at different granularities or introduce partially conflicting applicability conditions. We therefore construct an additional consolidation task over M independently sampled batches:

$$\mathcal{C}_0 = \mathcal{G}_{\text{merge}} \left(\mathcal{V}_0, \{ \Delta_{0,b}^{\text{edit}} \}_{b=1}^M \right), \quad (13)$$

where $\mathcal{C}_0$ is a globally consolidated editing target. The consolidation process removes semantic duplicates, merges compatible action patterns, reconciles overlapping trigger conditions, and preserves distinct skills when their preconditions or effects differ. This task teaches the learner to maintain library-level consistency beyond a single local update.

The complete Stage-0 dataset is

$$\mathcal{D}_0 = \mathcal{D}_0^{\text{gen}} \cup \mathcal{D}_0^{\text{edit}} \cup \mathcal{D}_0^{\text{con}}, \quad (14)$$

where the three subsets correspond to skill generation, library-conditioned editing, and cross-batch consolidation. Training on $\mathcal{D}_0$ updates the initial learner $\mathcal{S}_0$ into $\mathcal{S}_1$. The trained learner then evolves the initial library:

$$\Delta_0 = \mathcal{S}_1(\mathcal{V}_0, \mathcal{T}_0), \quad \mathcal{V}_1 = \text{Apply}(\mathcal{V}_0, \Delta_0). \quad (15)$$

Stage 1: Failure-Aware Learning Stage 0 exposes the learner only to successful oracle behavior. However, an on-line skill learner must also determine whether a failed execution reveals a missing action, an incorrect action or an invalid environmental condition. Stage 1 introduces trajectories generated by a heterogeneous pool of executor models, covering both successful and failed task executions.

Instead of randomly mixing trajectories from unrelated tasks, we construct each batch from multiple executors attempting the same task. For task q , the corresponding comparison batch is

$$B_{1,q} = \{\tau_{q,m} \mid m \in \mathcal{M}\}, \quad (16)$$

where $\mathcal{M}$ denotes the executor pool. Since all trajectories share the same instruction and success criterion, their behavioral differences provide controlled evidence for identifying generalizable skill content.

For each same-task batch, the teacher jointly analyzes all trajectories and produces a failure-aware editing target:

$$\Delta_{1,q}^* = \mathcal{G}(\mathcal{V}_1, B_{1,q}). \quad (17)$$

These examples constitute the Stage-1 dataset $\mathcal{D}_1$. Fine-tuning on $\mathcal{D}_1$ updates $\mathcal{S}_1$ into the failure-aware learner $\mathcal{S}_2$, which subsequently produces the next library:

$$\Delta_1 = \mathcal{S}_2(\mathcal{V}_1, \mathcal{T}_1), \quad \mathcal{V}_2 = \text{Apply}(\mathcal{V}_1, \Delta_1). \quad (18)$$

In summary, Stage 0 teaches the learner to perform structured library edits, while Stage 1 teaches it to identify invalid conditions and recovery strategies by contrasting successful and failed trajectories.

3.4 Skill OPD

Although curriculum SFT provides a strong initialization, its supervision is constructed from a fixed output distribution. We therefore introduce Skill OPD, adapting knowledge distillation (Hinton, Vinyals, and Dean 2015) and on-policy sequence distillation (Agarwal et al. 2024; Sheng et al. 2025; Li et al. 2026) to skill-library editing. Unlike offline SFT, Skill OPD trains the learner on contexts induced by its own generated outputs, reducing the train-inference distribution mismatch in autoregressive skill editing.

At OPD iteration i , we group incoming executor trajectories into editing batches. Conditioned on the current skill library $\mathcal{V}_i$ and a trajectory batch, the current learner $\mathcal{S}_i$ samples multi-turn editing rollouts from its own output distribution. Each rollout contains proposed edit operations together with the intermediate library states induced by applying and merging those operations.

A frozen stronger teacher computes token distributions on the same student-generated prefixes. Skill OPD does not use task rewards or policy-gradient updates; instead, it directly minimizes a top- K approximation to the forward KL divergence from the teacher to the student. Let c_ℓ denote a student-visited skill-editing context at token position ℓ , $p_T^{(K)}$ the teacher distribution restricted and renormalized over its top- K support, and $p_{\mathcal{S}_i}$ the student distribution. The OPD objective is

$$\mathcal{L}_{\text{OPD}}^{(i)} = \mathbb{E}_{c_\ell} \left[D_{\text{KL}} \left(p_T^{(K)}(\cdot \mid c_\ell) \parallel p_{\mathcal{S}_i}(\cdot \mid c_\ell) \right) \right]. \quad (19)$$

The expectation is taken over token contexts visited by the student’s own editing rollouts. Minimizing this objective updates the learner from $\mathcal{S}_i$ to $\mathcal{S}_{i+1}$. The improved learner is then used to regenerate the edits and produce the next skill library.

The updated library guides the executor in the next iteration, producing a new trajectory distribution for further skill-learner optimization. Skill OPD therefore forms a recursive loop in which the learner improves the library, and the evolving library continually supplies new on-policy experience for improving the learner.

4 Experiments

4.1 Experimental Setup

Benchmarks. We evaluate on 2 high-level embodied task-planning benchmarks from EmbodiedBench (Yang et al. 2025): EB-ALFRED and EB-Habitat. Both of them contain 6 splits that assess distinct capability dimensions: basic capabilities (Base), commonsense reasoning (Com.), complex instruction following (Comp.), visual recognition (Vis.), spatial reasoning (Spat.), and long-horizon planning (Long).

Implementation Details. We use Qwen3-VL-8B-Instruct (Bai et al. 2025) as the base model and Qwen3.7-Max (Alibaba Cloud 2026b) to construct supervision. Skill edits are generated from batches of 10 trajectories, and each consolidation step merges 15 candidate edits. SFT is performed for three epochs with a learning rate of 5×10^{-7} . For Skill OPD, we use Qwen3-VL-32B-Instruct (Bai et al. 2025) as the teacher and train for 20 epochs with a global batch size of 8 and one rollout per prompt. We apply top- K forward-KL distillation with $K = 32$ and no auxiliary policy-gradient loss.

Baselines. We select baselines to distinguish three sources of performance improvement: backbone capability (*No Skill*), direct executor post-training (*Training-based MLLMs*), and the use of knowledge derived from experience (*Experience-Augmented MLLMs*).

4.2 Main Results

As shown in Table 1, PRACTICE achieves the best overall performance among experience-augmented methods under the same Qwen3-VL-32B-Instruct executor. It obtains average success rates of 49.7% on EB-ALFRED and 58.3% on EB-Habitat, exceeding the strongest prior experience-augmented baseline. Among the reported split-level results, PRACTICE performs best on five EB-ALFRED splits and four EB-Habitat splits. The improvements are especially pronounced on Complex and Long in EB-ALFRED and on Complex and Visual in EB-Habitat, showing the benefit of converting experience into reusable guidance for complex instruction following and task breakdown. PRACTICE trails the strongest baselines on Base and Long in EB-Habitat, where highly variable target locations are less amenable to fixed action patterns, suggesting room for more adaptive search skills.

Some training-based methods, such as ELITE (Wei et al. 2026), achieve higher performance by post-training the executor on substantial task-specific data and interactions. In

Model / Method	EB-ALFRED							EB-Habitat						
	Avg.	Base	Common	Complex	Visual	Spatial	Long	Avg.	Base	Common	Complex	Visual	Spatial	Long
<i>No skill</i>														
GPT-5.4	65.3	76	72	76	54	58	56	65.7	98	52	62	62	40	80
Gemini-3-Flash	59.0	70	56	60	68	56	44	51.0	86	52	46	62	30	30
Qwen3.5-Flash	47.7	54	50	62	32	44	44	38.0	70	22	28	50	36	22
Qwen3-VL-32B-Instruct	24.3	30	38	34	28	16	0	37.3	70	34	38	32	32	18
<i>Training-based MLLMs</i>														
REBP (Wu et al. 2025)	35.6	54	42	46	28	38	6	20.0*	56	8	18	16	14	8
ORBIT† (Wu et al. 2026)	72.4†	86	62	82	72	60	—	22.4*	56	8	18	16	14	—
RoboGPT-R1 (Liu et al. 2025)	55.3	62	56	64	50	50	50	22.0*	64	8	18	20	12	10
RoboAgent (Xu et al. 2026)	67.0	72	48	64	78	60	80	22.3*	—	—	—	—	—	—
ESCA (Huang et al. 2025)	38.0	46	—	—	—	—	30	60.0	86	—	—	—	—	34
ELITE (Wei et al. 2026)	70.8	78	78	68	68	62	—	67.0	90	—	—	—	—	44
<i>Experience-Augmented MLLMs (Executor: Qwen3-VL-32B)</i>														
G-Mem (Zhang et al. 2025)	25.3	34	36	38	24	14	6	44.0	88	24	36	50	32	34
Mem0 (Chhikara et al. 2025)	30.3	40	42	40	26	26	8	38.3	84	24	26	42	34	20
A-Mem (Xu et al. 2025)	32.3	46	50	34	30	30	4	36.7	86	12	38	24	36	24
LangMem (LangChain 2025)	31.3	38	44	44	28	26	8	38.3	86	26	28	40	32	18
MemGen (Zhang et al. 2026)	14.2	22	18	16	6	20	4	25.7	50	14	22	36	22	10
EmbodiedSkill (Ju et al. 2026)	—	—	—	—	—	—	—	52.3	96	44	52	62	36	24
MemCompiler (Ding et al. 2026)	<u>40.0</u>	<u>52</u>	54	<u>48</u>	<u>38</u>	<u>38</u>	<u>10</u>	<u>55.7</u>	<u>90</u>	<u>58</u>	44	<u>64</u>	<u>40</u>	38
PRACTICE (Ours)	49.7	66	<u>50</u>	70	42	46	24	58.3	82	68	60	76	42	22
PRACTICE-GPT 5.4 (Ours)	<i>67.7</i>	<i>74</i>	<i>74</i>	<i>72</i>	<i>60</i>	<i>60</i>	<i>66</i>	<i>72.0</i>	<i>96</i>	<i>76</i>	<i>54</i>	<i>92</i>	<i>54</i>	<i>60</i>

Table 1: Success rate (%) on EB-ALFRED and EB-Habitat. † indicates that the average is computed over the five reported splits, excluding the Long split. * denotes unseen/OOD evaluation. “—” denotes results that are not reported. The best results are shown in **bold**, and the second-best results are underlined. PRACTICE results with GPT-5.4 executor (OpenAI 2026) are *italic* for distinction.

contrast, PRACTICE trains a separate skill learner while keeping the executor unchanged. It can substantially improve the task performances by continually converting new trajectories into refined skills without repeatedly updating the executor’s parameters.

Notably, PRACTICE-augmented Qwen3-VL-32B-Instruct surpasses the strong proprietary Qwen3.5-Flash, while PRACTICE-augmented GPT-5.4 achieves new state-of-the-art performance on EB-habitat benchmarks among all methods. These results show that our learner–library co-evolution pipeline can strengthen heterogeneous executors, narrowing the gap to stronger foundation models with a compact skill learner.

4.3 Ablation

Generalization across task executors. We additional conduct a cross-executor generalization study. Following the multi-executor evaluation protocol of MemCompiler (Ding et al. 2026), we instantiate the task executor with GPT-5.2, Gemini-3-Flash (Google 2025) (Gemini for shot), and Qwen3-VL-32B-Instruct (Qwen), covering both proprietary and open-weight models. For each executor, we compare skill-augmented performance against its matched unaugmented baseline under the same evaluation protocol.

As shown in Table 2, PRACTICE consistently improves all three executors. It increases the average success rate of GPT-5.4, GPT-5.2, Gemini and Qwen of 2.4, 30.3, 15.3, and 25.4 percentage points, respectively. Among the experience-

Method	Executor	No Aug.	With Aug.	Δ
BrainMem	Claude-3.5-Sonnet	64.0	74.7	+10.7
MemCtrl†	Gemma-3-12B	25.6	27.8	+2.2
MemCompiler	Qwen3-VL-32B	19.0	40.0	+21.0
LangMem	GPT-5.2	32.0	46.7	+14.7
	Gemini-3-Flash	59.0	78.0	+19.0
	Qwen3-VL-32B	19.0	31.3	+12.3
PRACTICE (Ours)	GPT-5.4	65.3	67.7	+2.4
	GPT-5.2	32.0	62.3	+30.3
	Gemini-3-Flash	59.0	74.3	+15.3
	Qwen3-VL-32B	24.3	49.7	+25.4

Table 2: Average success rate on EB-ALFRED. Δ denotes the absolute improvement from the experience augmentation. † indicates that the average is computed over the five reported splits, excluding the Visual split.

augmented methods shown with the same executor, PRACTICE yields the largest reported matched improvement on GPT-5.2 and Qwen. These results demonstrate the potential of improving diverse base executors through an evolving external skill library and support our claim that PRACTICE provides a general mechanism for sustained capability improvement without separately post-training each executor.

Effect of progressive skill-learner training. Table 3 presents a stage-wise ablation of PRACTICE’s training pipeline. The average success rate improves consistently

Setting	#Cards	Base	Com.	Comp.	Vis.	Spat.	Long	Avg.
No Skill	0	30	38	34	28	16	0	24.3
Initial Skill (BPE)	17	60	36	50	46	42	10	40.7
Stage 0 (SFT)	21	62	48	58	36	44	6	42.3
Stage 1 (SFT)	17	66	46	60	40	46	14	45.3
Stage 2 (OPD)	29	66	50	70	42	46	24	49.7

Table 3: Ablation of PRACTICE’s progressive training pipeline on EB-ALFRED. We report task success rate (SR, %) across three training stages: curriculum SFT (Stages 0–1) and online skill-edit distillation (Stage 2). **Executor**: Qwen3-VL-32B-Instruct; **Student**: Qwen3-VL-8B-Instruct. The best result in each column is shown in **bold**.

across all subsequent training stages. Stage 0 increases the average from 40.7% to 42.3% by teaching the learner basic skill generation and consolidation from successful oracle trajectories. Introducing failure-aware skill editing in Stage 1 further raises performance to 45.3%, with a particularly clear improvement on the Long split from 6% to 14%. Finally, Stage 2 achieves the largest incremental gain among the learned stages, improving the average success rate by 4.4 points. At the same time, the number of skill cards in this stage also grows to the greatest one with 29 cards. These results indicate that during Skill OPD stage, skill learner effectively learn to extract more new and effective skills by aligning with the stronger teacher on the skill-edit distribution, reducing the discrepancy between offline supervision and the edits encountered during iterative library evolution.

Effect of failure-aware skill learning. To isolate the contribution of failure-aware supervision, we compare two variants of Stage 1. The full variant (Failure-Aware) uses both successful and failed trajectories, requiring the skill learner to contrast their action sequences and environment feedback to identify invalid conditions and derive corresponding recovery strategies. These findings are recorded in the respective fields of each skill card. As a control, Stage 1 (Succ-Only) uses only successful trajectories and removes supervision for these two failure-related fields while retaining the rest of the skill-learning procedure.

As shown in Table 4, simply adding more successful trajectories provides only a marginal improvement, increasing the average success rate from 42.3% to 42.7%. In contrast, failure-aware skill learning raises the average to 45.3%, outperforming the success-only variant by 2.6 percentage points and Stage 0 by 3.0 points. The largest improvement appears on the Long split, where success increases from 8% to 14%. Failure-aware learning also improves Base and Complex by 4 points and Spatial by 2 points over the success-only control. These results indicate that failed trajectories provide supervision that cannot be obtained merely by collecting additional successful examples: they expose invalid action patterns, violated preconditions, and corrective behaviors that help the executor recover during complex and long-horizon tasks.

Figure 3 illustrates how failure-aware skills recovery from execution errors. After the initial pickup fails, the executor matches the environment feedback to an invalid condition recorded in the skill card and invokes the corresponding

Variant	Base	Com.	Comp.	Vis.	Spat.	Long	Avg.
Stage 0	62	48	58	36	44	6	42.3
Stage 1 (Succ-Only)	62	46	56	40	44	8	42.7
Stage 1 (Failure-Aware)	66	46	60	40	46	14	45.3
Δ	+4	+0	+4	+0	+2	+6	+2.6

Table 4: Ablation study of failure-aware recovery learning on EB-ALFRED. All results are SR, %. Δ denotes the improvement of Stage 1 (Failure-Aware) over Stage 1 (Succ-Only).

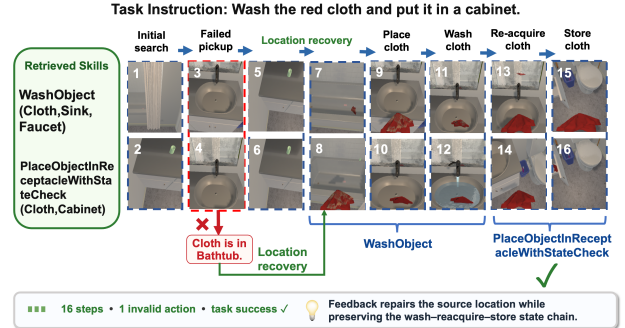


Figure 3: **A representative failure-recovery trajectory.** The **PickUp** failure triggered “object not found” strategy redirects the search to an alternative location. The executor then resumes the task and completes it using **WashObject** and **PlaceObjectReceptacle** in 16 steps.

recovery action. This example demonstrates that learned recovery strategies make skill-guided execution more robust to unexpected environment states.

5 Conclusion

This work reframes sustained improvement in embodied task planning as a problem of learning how interaction experience should be transformed into persistent procedural knowledge. PRACTICE shows that reusable action patterns become substantially more effective when paired with explicit applicability conditions and failure-recovery strategies: together, they help executors decompose complex instructions, avoid repeated failure modes, and recover when execution deviates from the expected procedure. Rather than relying on a fixed, prompt-engineered maintenance workflow, PRACTICE learns this transformation through a co-evolution of learner–library but with a frozen executor: the updated skill library shapes future trajectories, while those trajectories improve both the skill refinement policy and the knowledge retained in the next library. The consistent gains across training stages and heterogeneous executors validate the effectiveness and generality of this cycle. More broadly, our results suggest that learning to maintain an evolving procedural layer offers an effective path complementary to post-training the executor itself. Sustained capability growth can therefore arise not only from improving how an agent acts, but also from improving how it turns past experiences into reusable knowledge for future decisions.

References

- Agarwal, R.; Vieillard, N.; Zhou, Y.; Stanczyk, P.; Ramos Garea, S.; Geist, M.; and Bachem, O. 2024. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes. In *International Conference on Learning Representations (ICLR)*.
- Ahn, M.; Brohan, A.; Brown, N.; Chebotar, Y.; Cortes, O.; David, B.; Finn, C.; Fu, C.; Gopalakrishnan, K.; Hausman, K.; Herzog, A.; Ho, D.; Hsu, J.; Ibarz, J.; Ichter, B.; Irpan, A.; Jang, E.; Ruano, R. J.; Jeffrey, K.; Jesmonth, S.; Joshi, N. J.; Julian, R.; Kalashnikov, D.; Kuang, Y.; Lee, K.-H.; Levine, S.; Lu, Y.; Luu, L.; Parada, C.; Pastor, P.; Quiambao, J.; Rao, K.; Rettinghouse, J.; Reyes, D.; Sermanet, P.; Sievers, N.; Tan, C.; Toshev, A.; Vanhoucke, V.; Xia, F.; Xiao, T.; Xu, P.; Xu, S.; Yan, M.; and Zeng, A. 2022. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. *arXiv preprint arXiv:2204.01691*.
- Alibaba Cloud. 2026a. Qwen3.5-Flash. <https://www.alibabacloud.com/help/en/model-studio/vision-model/>. Model snapshot qwen3.5-flash-2026-02-23; accessed July 30, 2026.
- Alibaba Cloud. 2026b. Qwen3.7-Max. <https://www.alibabacloud.com/help/en/model-studio/text-generation-model/>. Alibaba Cloud Model Studio documentation. Accessed: 2026-07-29.
- Bai, S.; Cai, Y.; Chen, R.; Chen, K.; Chen, X.; Cheng, Z.; Deng, L.; Ding, W.; Gao, C.; Ge, C.; et al. 2025. Qwen3-VL Technical Report. *arXiv preprint arXiv:2511.21631*.
- Bengio, Y.; Louradour, J.; Collobert, R.; and Weston, J. 2009. Curriculum Learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, 41–48.
- Chen, H.; Zhao, M.; Yang, R.; Ma, Q.; Yang, K.; Yao, J.; Wang, K.; Bai, H.; Wang, Z.; Pan, R.; Zhang, M.; Barreiros, J.; Onol, A.; Zhai, C.; Ji, H.; Li, M.; Zhang, H.; and Zhang, T. 2025. ERA: Transforming VLMs into Embodied Agents via Embodied Prior Learning and Online Reinforcement Learning. *arXiv:2510.12693*.
- Chhikara, P.; Khant, D.; Aryan, S.; Singh, T.; and Yadav, D. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv preprint arXiv:2504.19413*.
- Comanici, G.; Bieber, E.; Schaeckermann, M.; et al. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. *arXiv preprint arXiv:2507.06261*.
- Ding, X.; Wang, X.; Yang, Y.; Wu, H.; Jiang, S.; Zhang, Q.; Mi, L.; Zhu, H.; Li, K.; Liu, Y.; et al. 2026. MemCompiler: Compile, Don't Inject—State-Conditioned Memory for Embodied Agents. *arXiv preprint arXiv:2605.07594*.
- Dorbala, V. S.; and Manocha, D. 2026. MemCtrl: Using MLLMs as Active Memory Controllers on Embodied Agents. *arXiv preprint arXiv:2601.20831*.
- Google. 2025. Gemini 3 Flash Preview. <https://ai.google.dev/gemini-api/docs/models/gemini-3-flash-preview>. Accessed: 2026-07-29.
- Google DeepMind. 2025. Gemini 3 Flash Model Card. <https://deepmind.google/models/model-cards/gemini-3-flash/>. Published December 17, 2025.
- Google DeepMind. 2026a. Gemini 3.1 Pro Model Card. <https://deepmind.google/models/model-cards/gemini-3-1-pro/>. Published February 19, 2026.
- Google DeepMind. 2026b. Gemini 3.5 Flash Model Card. <https://deepmind.google/models/model-cards/gemini-3-5-flash/>. Published May 19, 2026.
- Hinton, G.; Vinyals, O.; and Dean, J. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*.
- Huang, J.; Sethi, A.; Kuo, M.; Keoliya, M.; Velingker, N.; Jung, J.; Lim, S. N.; Li, Z.; and Naik, M. 2025. ESCA: Contextualizing Embodied Agents via Scene-Graph Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, 2828–2870.
- Jiang, G.; Su, Z.; Qu, X.; and Fung, Y. R. 2026. XSkill: Continual Learning from Experience and Skills in Multimodal Agents. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*.
- Ju, R.; Wang, X.; Ding, X.; Yang, Y.; Wu, H.; Jiang, S.; Zhang, Q.; Wen, H.; Li, X.; Wang, W.; et al. 2026. EmbodiSkill: Skill-Aware Reflection for Self-Evolving Embodied Agents. *arXiv preprint arXiv:2605.10332*.
- LangChain. 2025. LangMem. <https://github.com/langchain-ai/langmem>. Accessed: 2026-05-04.
- Li, Y.; Zuo, Y.; He, B.; Zhang, J.; Xiao, C.; Qian, C.; Yu, T.; Gao, H.-a.; Yang, W.; Liu, Z.; and Ding, N. 2026. Rethinking On-Policy Distillation of Large Language Models: Phenomenology, Mechanism, and Recipe. *arXiv preprint arXiv:2604.13016*.
- Liu, J.; Nie, B.; Li, B.; Chen, Y.; Wang, Y.; He, S.; and Li, H. 2025. RoboGPT-R1: Enhancing Robot Task Planning with Reinforcement Learning. *arXiv preprint arXiv:2510.14828*.
- Ni, J.; Liu, Y.; Liu, X.; Sun, Y.; Zhou, M.; Cheng, P.; Wang, D.; Zhao, E.; Jiang, X.; and Jiang, G. 2026. Trace2Skill: Distill Trajectory-Local Lessons into Transferable Agent Skills. *arXiv:2603.25158*.
- OpenAI. 2026. Introducing GPT-5.4. <https://openai.com/index/introducing-gpt-5-4/>. Published March 5, 2026.
- Ouyang, S.; Yan, J.; Chen, Y.; Han, R.; Wang, Z.; Mishra, B. D.; Meng, R.; Li, C.-L.; Jiao, Y.; Zha, K.; et al. 2026. SkillIOS: Learning skill curation for self-evolving agents. *arXiv preprint arXiv:2605.06614*.
- Sennrich, R.; Haddow, B.; and Birch, A. 2016. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 1715–1725.
- Sheng, G.; Zhang, C.; Ye, Z.; Wu, X.; Zhang, W.; Zhang, R.; Peng, Y.; Lin, H.; and Wu, C. 2025. HybridFlow: A Flexible and Efficient RLHF Framework. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, 1279–1297. ACM.
- Song, C. H.; Wu, J.; Washington, C.; Sadler, B. M.; Chao, W.-L.; and Su, Y. 2023. LLM-Planner: Few-Shot Grounded

Planning for Embodied Agents with Large Language Models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2998–3009.

Wang, K.; Zhang, P.; Wang, Z.; Gao, Y.; Li, L.; Wang, Q.; Chen, H.; Lu, Y.; Yang, Z.; Wang, L.; Krishna, R.; Wu, J.; Li, F.-F.; Choi, Y.; and Li, M. 2025. VAGEN: Reinforcing World Model Reasoning for Multi-Turn VLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38.

Wei, B.; Xia, Z.; Liu, D.; Zhou, X.; Lin, Z.; and Wang, Y. 2026. ELITE: Experiential Learning and Intent-Aware Transfer for Self-improving Embodied Agents. *arXiv preprint arXiv:2603.24018*.

Wu, D.; Fan, J.; Gu, C.; Wang, G.; Yin, W.; Li, W.; and Jin, B. 2026. On-Policy Reinforcement Fine-Tuning with Offline Reward for Multi-Step Embodied Planning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL)*, 39277–39307. San Diego, California, United States: Association for Computational Linguistics.

Wu, D.; Fan, J.; Zang, J.; Wang, G.; Yin, W.; Li, W.; and Jin, B. 2025. Reinforced Reasoning for Embodied Planning. *arXiv preprint arXiv:2505.22050*.

Xu, P.; Zheng, J.; and Mu, Y. 2026. RoboAgent: Chaining Basic Capabilities for Embodied Task Planning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 15276–15290.

Xu, W.; Liang, Z.; Mei, K.; Gao, H.; Tan, J.; and Zhang, Y. 2025. A-MEM: Agentic Memory for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38.

Yang, R.; Chen, H.; Zhang, J.; Zhao, M.; Qian, C.; Wang, K.; Wang, Q.; Koripella, T. V.; Movahedi, M.; Li, M.; Ji, H.; Zhang, H.; and Zhang, T. 2025. EmbodiedBench: Comprehensive Benchmarking Multi-modal Large Language Models for Vision-Driven Embodied Agents. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267, 70576–70631. PMLR.

Yang, Y.; Li, J.; Pan, Q.; Zhan, B.; Cai, Y.; Du, L.; Zhou, J.; Chen, K.; Chen, Q.; Li, X.; Zhang, B.; and He, L. 2026. AutoSkill: Experience-Driven Lifelong Learning via Skill Self-Evolution. *arXiv:2603.01145*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.

Yuan, H.; Bai, Y.; Fu, Y.; Zhou, B.; Feng, Y.; Xu, X.; Zhan, Y.; Karlsson, B. F.; and Lu, Z. 2025. Being-0: A Humanoid Robotic Agent with Vision-Language Models and Modular Skills. *arXiv preprint arXiv:2503.12533*.

Zhai, Y.; Bai, H.; Lin, Z.; Pan, J.; Tong, S.; Zhou, Y.; Suhr, A.; Xie, S.; LeCun, Y.; Ma, Y.; and Levine, S. 2024. Fine-Tuning Large Vision-Language Models as Decision-Making Agents via Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 110935–110971.

Benchmark	Stage	Succ.	Fail.	# Gen/Edit	# Conso.	Total
EB-ALFRED	Stage 0A	300	0	1,000	400	1,400
	Stage 0B	300	0	400	0	400
	Stage 1	1,285	1,123	300	150	450
	Subtotal	1,885	1,123	1,700	550	2,250
EB-Habitat	Stage 0A	300	0	1,000	400	1,400
	Stage 0B	150	0	300	100	400
	Stage 1	1,316	1,084	300	150	450
	Subtotal	1,766	1,084	1,700	550	2,250

Table 5: Statistics of the final curriculum SFT datasets. Succ. and Fail. count source trajectories before batching, whereas Edit, Consolidation (Conso.), and Total count the resulting SFT samples.

Zhang, G.; Fu, M.; Wang, K.; Wan, F.; Yu, M.; and Yan, S. 2025. G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38.

Zhang, G.; Fu, M.; and Yan, S. 2026. MemGen: Weaving Generative Latent Memory for Self-Evolving Agents. In *International Conference on Learning Representations (ICLR)*.

A Additional Method Details

A.1 Training Data Construction

Our curriculum supervised fine-tuning (SFT) consists of two stages. Stage 0 uses only successful oracle trajectories to establish the learner’s basic skill-generation, skill-edit and skill-consolidate capabilities. For generation (Stage 0A), the `current_skill_pool` is set to an empty list. We randomly sample $N = 10$ trajectories as a batch from 300 oracle trajectories in both EB-ALFRED and EB-Habitat to construct supervision from *Qwen-3.7-max* (Bai et al. 2025). Therefore we get several generated skill candidates for each batch, after that, we randomly sample $M = 15$ candidates to construct skill-consolidate data. For editing (Stage 0B), the `current_skill_pool` is the initial skill library V_0 . Similarly as Stage 0A, we generate skill editing data from randomly trajectories batches from oracle.

Stage 1 groups trajectories produced by 8 heterogeneous executors according to the task: *Gemini-2.5-Flash* (Comanici et al. 2025), *Gemini-3-Flash* (Google DeepMind 2025), *Gemini-3.1-Pro* (Google DeepMind 2026a), *Gemini-3.5-Flash* (Google DeepMind 2026b), *GPT-5.4* (OpenAI 2026), *Qwen3.5-Flash* (Alibaba Cloud 2026a), and *Qwen3-VL-8B/32B-Instruct* (Bai et al. 2025). By contrasting successful and failed executions of the same task, the learner generates failure-aware edits that capture additional `invalid_conditions` and `recovery_strategies`. Then from each group, we collect the corresponding skill editing with the skill pool of V_1 , and consolidation data after that.

In Table 5, skill-generation and local library-editing examples are jointly counted as *edit samples*, while cross-batch consolidation examples are counted as *merge samples*.

Algorithm 1: Progressive Learner–Library Co-Evolution in PRACTICE.

Require: Frozen task executor π_{exec} ; initial skill learner $\mathcal{S}_0$; oracle trajectories $\mathcal{T}^{\text{ora}}$; heterogeneous executors Π_{het} ; SFT supervision teacher P_{sup} ; OPD teacher P_T ; OPD rounds R

Ensure: Trained skill learner $\mathcal{S}_{R+1}$ and evolved skill library $\mathcal{V}_{R+1}$

- 1: **// Initial skill-library construction**
- 2: $\mathcal{P}_0 \leftarrow \text{BPE-PATTERNDISCOVERY}(\mathcal{T}^{\text{ora}})$
- 3: $\mathcal{V}_0 \leftarrow \text{INITIALIZESKILLLIBRARY}(\mathcal{S}_0, \mathcal{P}_0, \mathcal{T}^{\text{ora}})$
- 4: **// Stage 0: Oracle-Grounded SFT**
- 5: $\mathcal{D}_0 \leftarrow \text{CONSTRUCTORACLESUPERVISION}(P_{\text{sup}}, \mathcal{V}_0, \mathcal{T}^{\text{ora}})$
- 6: $\mathcal{S}_1 \leftarrow \text{SUPERVISEDFINETUNE}(\mathcal{S}_0, \mathcal{D}_0)$ {Learner update}
- 7: $\mathcal{T}_0 \leftarrow \text{ROLLOUT}(\pi_{\text{exec}}, \mathcal{V}_0)$
- 8: $\mathcal{V}_1 \leftarrow \text{UPDATELIBRARY}(\mathcal{S}_1, \mathcal{V}_0, \mathcal{T}_0)$ {Library update}
- 9: **// Stage 1: Failure-Aware SFT**
- 10: $\mathcal{T}_1 \leftarrow \text{COLLECTSAMETASKROLLOUTS}(\Pi_{\text{het}}, \mathcal{V}_1)$
- 11: $\mathcal{B}_1 \leftarrow \text{GROUPBYTASK}(\mathcal{T}_1)$
- 12: $\mathcal{D}_1 \leftarrow \text{CONSTRUCTFAILUREAWARESUPERVISION}(P_{\text{sup}}, \mathcal{V}_1, \mathcal{B}_1)$
- 13: $\mathcal{S}_2 \leftarrow \text{SUPERVISEDFINETUNE}(\mathcal{S}_1, \mathcal{D}_1)$ {Learner update}
- 14: $\mathcal{V}_2 \leftarrow \text{UPDATELIBRARY}(\mathcal{S}_2, \mathcal{V}_1, \mathcal{T}_1)$ {Library update}
- 15: **// Stage 2: On-Policy Distillation**
- 16: **for** $i = 2, \dots, R$ **do**
- 17: $\mathcal{T}_i \leftarrow \text{ROLLOUT}(\pi_{\text{exec}}, \mathcal{V}_i)$
- 18: $\mathcal{D}_i^{\text{OPD}} \leftarrow \text{CONSTRUCTOPDPROMPTS}(\mathcal{V}_i, \mathcal{T}_i)$
- 19: $\mathcal{S}_{i+1} \leftarrow \text{ONPOLICYDISTILLATION}(\mathcal{S}_i, P_T, \mathcal{D}_i^{\text{OPD}})$ {Algorithm 2}
- 20: $\mathcal{V}_{i+1} \leftarrow \text{UPDATELIBRARY}(\mathcal{S}_{i+1}, \mathcal{V}_i, \mathcal{T}_i)$ {Library update}
- 21: **end for**
- 22: **return** $\mathcal{S}_{R+1}, \mathcal{V}_{R+1}$
- 23: **// Subroutine: UpdateLibrary**($\mathcal{S}, \mathcal{V}, \mathcal{T}$)
- 24: Partition trajectories into batches $\{\mathcal{B}_m\}_{m=1}^M$
- 25: **for** $m = 1, \dots, M$ **do**
- 26: $\Delta_m \leftarrow \mathcal{S}(\mathcal{V}, \mathcal{B}_m)$
- 27: $\Delta_m \in \{\text{ADD}, \text{REVISE}, \text{MERGE}, \text{REMOVE}\}$
- 28: **end for**
- 29: $\tilde{\mathcal{V}} \leftarrow \text{APPLYEDITS}(\mathcal{V}, \bigcup_{m=1}^M \Delta_m)$
- 30: $\mathcal{V}' \leftarrow \text{HIERARCHICALCONSOLIDATION}(\tilde{\mathcal{V}})$
- 31: **return** $\mathcal{V}'$

A.2 Training Algorithm

Algorithm 1 presents the pseudocode for the overall PRACTICE framework. Specifically, we implement the Stage 2 training using the on-policy distillation (OPD) pipeline in `ver1`. The training data are prompt-only: given a trajectory prompt, the current student first generates a skill update from its own policy. A frozen teacher then evaluates the student-generated sequence and provides its Top- K token distribution at each response position. We directly optimize the student using a Top- K approximation of the forward KL divergence. No task reward or policy-gradient objective is used.

Let π_θ denote the student skill learner and π_T the frozen

teacher. For a prompt x , the student first samples

$$y \sim \pi_\theta(\cdot | x), \quad (20)$$

where $y = (y_1, \dots, y_T)$ is the student-generated skill update. At each response position t , the teacher is evaluated on the student-induced context $s_t = (x, y_{<t})$ and returns its Top- K token set

$$\mathcal{K}_t = \text{TopK}_K(\pi_T(\cdot | s_t)). \quad (21)$$

We minimize the truncated forward-KL objective

$$\begin{aligned} \ell_t &= \sum_{v \in \mathcal{K}_t} \pi_T(v | s_t) \\ &\times [\log \pi_T(v | s_t) - \log \pi_\theta(v | s_t)]. \end{aligned} \quad (22)$$

The loss is aggregated only over valid student-response tokens:

$$\mathcal{L}_{\text{OPD}} = \frac{\sum_{t=1}^T m_t \ell_t}{\sum_{t=1}^T m_t}, \quad (23)$$

where m_t is the response-token mask. In our implementation, $K = 32$. The student is updated by direct backpropagation through $\mathcal{L}_{\text{OPD}}$, while the teacher remains frozen.

Unlike policy-gradient OPD, our configuration directly backpropagates the distribution-matching objective through the student probabilities. Task rewards and PPO/GRPO policy-gradient losses are disabled; hence the total optimization objective is simply

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{OPD}}. \quad (24)$$

Because each new response is sampled from the latest student before teacher scoring, the distillation states evolve together with the student policy, rather than remaining fixed to an offline teacher-generated dataset.

B Implementation Details

We first train Qwen3-VL-8B-Instruct with curriculum SFT using supervision constructed by Qwen3.7-Max. The resulting SFT checkpoint initializes SKILL OPD, in which a fixed Qwen3-VL-32B-Instruct teacher provides token-level soft targets for the editing rollouts generated by the current student policy. We use pure Top- K forward-KL distillation without auxiliary reward or policy-gradient objectives. The complete data-generation, optimization, batching, sequence-length, and system configurations are reported in Table 6. The training time for SFT stages are about 7 GPU hours in total and 64 GPU hours for OPD. So overall, the method is computationally lightweight.

C Skill Analysis

C.1 What We Learned

We analyze the action-sequence lengths of the skill libraries produced by the skill learner after all three training stages on both EB-ALFRED and EB-Habitat. As shown in Figure 4-5, we visualize the ten longest skills for each benchmark. On EB-ALFRED, the learned library contains 29 skills, comprising 13 single-action skills and 16 composite skills. On EB-Habitat, it contains 15 skills, including 5 single-action skills and 10 composite skills.

Algorithm 2: On-Policy Distillation for the Skill Learner

Require: Prompt-only dataset $\mathcal{D}$; student π_θ ; frozen teacher π_T ; Top- K size K ; chunk size C

Ensure: Distilled student π_θ

```

1: for each training step do
2:   Sample prompt batch  $\mathcal{B} = \{x_i\}_{i=1}^B$  from  $\mathcal{D}$ 
3:   // On-policy student rollout
4:   for each  $x_i \in \mathcal{B}$  do
5:     Sample  $y_i \sim \pi_\theta(\cdot | x_i)$ 
6:     Form sequence  $z_i = [x_i; y_i]$ 
7:     Construct response mask  $m_i$ 
8:   end for
9:   // Frozen-teacher supervision
10:  for each student-generated sequence  $z_i$  do
11:    Evaluate frozen  $\pi_T$  on  $z_i$ 
12:    for each response position  $t$  do
13:      Set context  $s_{i,t} = (x_i, y_{i,<t})$ 
14:      Obtain teacher Top- $K$  set  $\mathcal{K}_{i,t}$ 
15:      Store log  $\pi_T(v | s_{i,t})$  for  $v \in \mathcal{K}_{i,t}$ 
16:    end for
17:  end for
18:  // Student distribution matching
19:  Forward  $\pi_\theta$  on  $\{z_i\}_{i=1}^B$ 
20:  for response positions in chunks of size  $C$  do
21:    Compute student log-normalizers
22:    Gather logits at teacher Top- $K$  indices
23:    Obtain log  $\pi_\theta(v | s_{i,t})$  for  $v \in \mathcal{K}_{i,t}$ 
24:  end for
25:  Clip teacher and student log-probabilities from below at  $-10$ 
26:  // Top- $K$  forward-KL distillation
27:  for each valid response position  $(i, t)$  do
28:    Compute  $\ell_{i,t}$  using Eq. (22)
29:    Clip  $\ell_{i,t}$  to  $[0, 10]$ 
30:  end for
31:  Aggregate token losses using Eq. (23)
32:  Set  $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{OPD}}$ 
33:  Update  $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{total}}$ 
34:  Synchronize student weights to the rollout engine
35: end for

```

The learner discovers reusable procedures at substantially different levels of complexity. On EB-ALFRED, `HeatAndStoreFoodItem` expands into 15 actions, integrating object retrieval, appliance operation, heating, and final storage. On EB-Habitat, `MultiObjectSearchAndTransfer` contains 8 actions and captures repeated object search, retrieval, and delivery. These examples demonstrate that PRACTICE can abstract complete long-horizon procedures. Such long macros can simplify planning, especially those complex long-horizon tasks.

As shown in the Figure 6, both libraries exhibit a clear hierarchy between primitive control and procedural abstraction. In EB-ALFRED, 11 of the 16 composite skills contain 4 to 7 actions. In EB-Habitat, 9 of the 10 composite skills contain 2 to 4 actions, with only one extending to 8 actions. This difference indicates that PRACTICE adapts skill granularity to

Hyperparameter	Value
<i>Curriculum SFT</i>	
Student model	Qwen3-VL-8B-Instruct
Supervision model	Qwen3.7-Max
Training epochs	3
Optimizer	AdamW
Learning rate	5×10^{-7}
Learning-rate schedule	Cosine
Weight decay	0.01
Gradient clipping	1.0
Maximum sequence length	8,192
Training precision	bfloat16
Stage 0 Training time	~ 4 GPU hours
Stage 1 Training time	~ 3 GPU hours
<i>Skill OPD</i>	
Student model	Stage 1 SFT checkpoint
Teacher model	Qwen3-VL-32B-Instruct
Teacher update	Frozen
Distillation objective	Top- K forward KL
Teacher Top- K	32
Top- K chunk size	8
Distillation loss coefficient	1.0
Maximum loss clipping	10
Minimum log-prob. clipping	-10
Training epochs	6
Training time	~ 64 GPU hours
Global rollout batch size	8
Rollouts per prompt N	1
Optimization mini-batch size	8
Optimization epochs per rollout batch	1
Policy updates per rollout batch	1
Per-GPU micro-batch size	1
Optimizer	AdamW
Learning rate	5×10^{-7}
Learning-rate schedule	Constant
Weight decay	0.01
Gradient clipping	1.0
Per-GPU token budget	12,289
Student training resources	FSDP on $8 \times$ A100-40GB
Student rollout	vLLM Colocated, TP = 2
Teacher serving resources	vLLM on $8 \times$ A100-40GB, TP = 8
Rollout precision	bfloat16
vLLM memory utilization	0.6

Table 6: Training hyperparameters and system configurations for curriculum SFT and *Skill OPD*.

the procedural structure of each environment: EB-ALFRED requires longer manipulation routines, whereas EB-Habitat primarily benefits from compact search-and-transfer abstractions.

C.2 Skill Execution Analysis

We analyze how the same executor behaves with and without the learned skill library. Figure 7 summarizes all 300 EB-

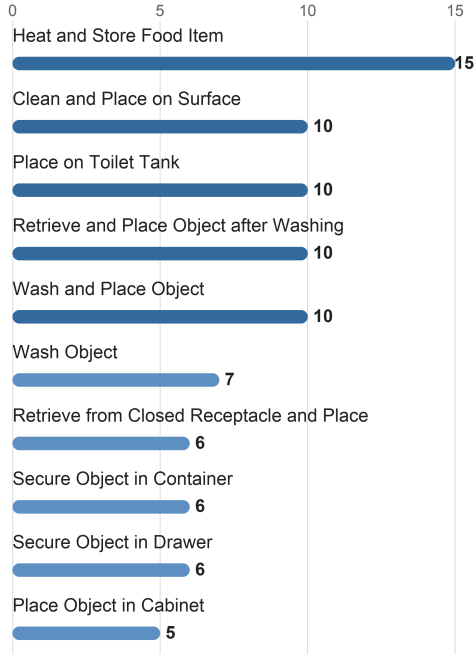


Figure 4: The 10 skills with the longest default action sequences in the EB-ALFRED skill library.

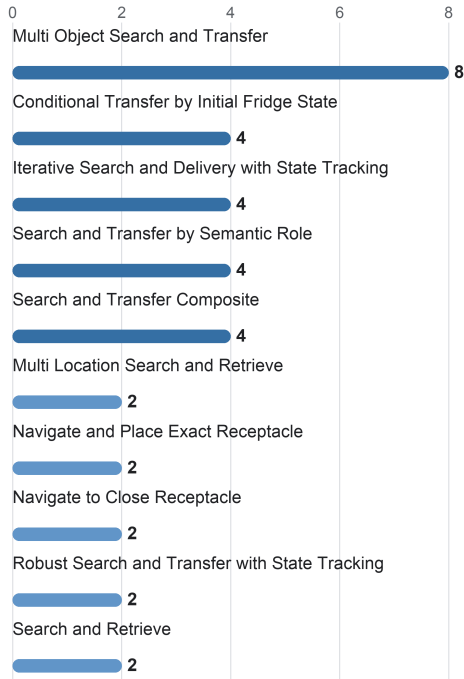


Figure 5: The 10 skills with the longest default action sequences in the EB-Habitat skill library.

ALFRED episodes. First, learned skills enable more failure-free execution. The number of episodes without any rejected

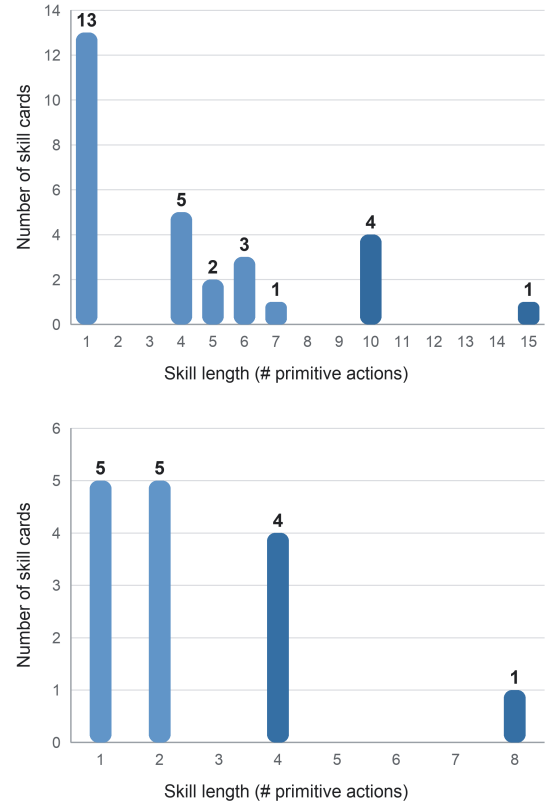


Figure 6: Distribution of default skill-expansion lengths in the learned skill libraries for (a) EB-ALFRED and (b) EB-Habitat. Skill length is measured by the number of primitive actions in the default expansion pattern.

action increases from 40/300 (13.3%) to 77/300 (25.7%), while successful failure-free episodes increase from 38/300 (12.7%) to 73/300 (24.3%). This indicates that the learned skills help the executor generate more executable plans that satisfy action preconditions.

Second, learned skills substantially improve outcomes even when rejected actions still occur. Although the number of episodes containing a rejected action decreases from 260 to 223, successful episodes within this branch increase from 35/300 (11.7%) to 76/300 (25.3%). The lower population-level percentages for different follow-up actions and immediate local recovery mainly reflect that fewer episodes enter the failure branch; they should not be interpreted as conditional recovery rates.

Overall, the improvement is jointly explained by proactive failure avoidance and post-failure recovery. These results show that PRACTICE improves embodied task execution not only by producing more valid plans, but also by helping the executor preserve progress and ultimately complete the task after execution failures.

We further analyze the execution situation of each skills. The Figure 8 illustrates the relationship between skill usage frequency and task-level benefit. The x-axis shows the number of skill invocations, while the y-axis reports the paired

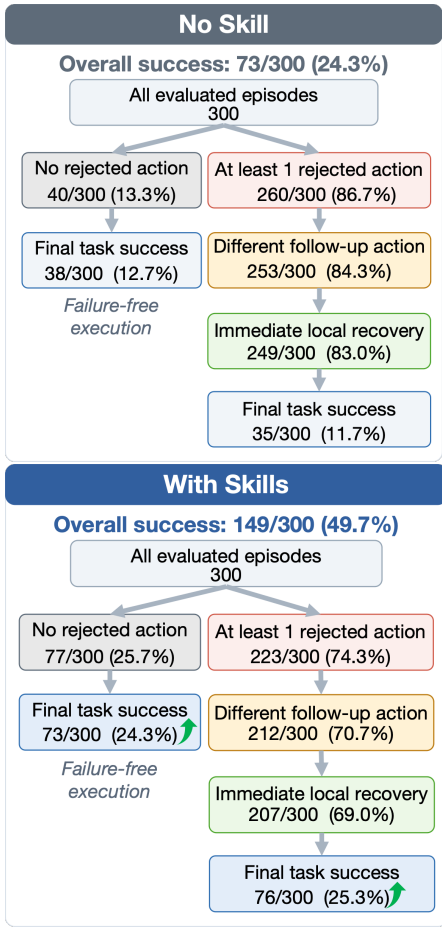


Figure 7: Execution outcomes with and without learned skills over 300 EB-ALFRED episodes.

success-rate improvement over the no-skill 0-shot baseline on the same episodes. Bubble size represents task coverage, and error bars indicate 95% bootstrap confidence intervals. Compared with success rates conditioned only on skill invocation, this paired comparison better controls for episode difficulty and reduces selection bias.

Overall, the PRACTICE method improves the task success rate from 24.3% to 40.0%. TurnOff, Close, TurnOn, Open, PutDown, Find, and Pickup all show consistent positive improvements. In particular, Find, Pickup, and PutDown combine high invocation frequency with broad task coverage, demonstrating strong generalizability. TurnOff, Close, and TurnOn achieve larger gains, suggesting that skill cards effectively stabilize interaction procedures that are otherwise error-prone. MoveObjectToReceptacle and WashObject also show large improvements, although their limited coverage requires further validation.

The four quadrants represent distinct types of skills. The upper-right quadrant contains frequent, broadly applicable, and effective skills, which form the core strength of the learned skill library. The upper-left quadrant contains

promising specialized skills with insufficient coverage and therefore calls for additional evaluation data. The lower-right quadrant represents frequently used but harmful skills; this region is nearly empty, indicating that high-frequency skills do not introduce systematic negative effects. The lower-left quadrant mainly contains low-frequency skills such as Drop and HoldObjectWhileTurningOnLight, whose negative estimates remain uncertain because of limited samples. Skills such as PlaceObjectInContainer, PlaceObjectOnNonContainerSurface, TurnOffLight, and SliceWithTool do not show robust improvements and should be prioritized for expansion or routing refinement.

Overall, the learned skill library improves success while concentrating model usage on broadly reusable skills with stable positive returns. These results demonstrate that the proposed method can extract transferable behavioral patterns from trajectories and improve execution reliability across diverse tasks.

C.3 Skill Usage in Different Splits

We compare the PRACTICE with the No-Skill baseline through paired evaluation through six different splits. Each episode is categorized as both agents succeeding, only the PRACTICE succeeding, only the No-Skill agent succeeding, or both agents failing.

As shown in Figure 9-10, the PRACTICE provides consistent gains on both benchmarks. On EB-ALFRED, skills recover 91 episodes that the No-Skill baseline fails, while introducing regressions on only 15 episodes, resulting in a net improvement of $\Delta\text{SR} = +25.3$ percentage points. The largest gains appear on base and complex-instruction tasks (+36 points), followed by spatial tasks (+30 points). On long-horizon tasks, the PRACTICE solves 12 episodes while the baseline solves none. These results indicate that reusable skills effectively translate diverse instructions into stable multi-step action sequences.

On EB-Habitat, skills recover 58 failed episodes while causing 32 regressions, producing an overall improvement of $\Delta\text{SR} = +8.7$ points. The largest gain occurs on visual-appearance tasks (+20 points), followed by complex-instruction (+12 points) and base tasks (+10 points). The smaller improvement on EB-Habitat reflects its stronger dependence on scene exploration and object localization: navigation is restricted to receptacles, so successful execution often requires locating an object indirectly before applying a reusable manipulation pattern. Spatial tasks show no net improvement, while long-horizon tasks improve by only 4 points, suggesting that spatial grounding and extended state tracking remain difficult to encode through fixed skill structures alone.

Overall, skills provide a substantial paired advantage on both benchmarks. Their primary benefit is reducing the planning complexity of recurring multi-step behaviors and improving execution consistency. Remaining failures are concentrated in tasks requiring visual grounding, indirect object search, spatial reasoning, and long-horizon coordination, highlighting the need for stronger adaptive skill selection and state-aware composition.

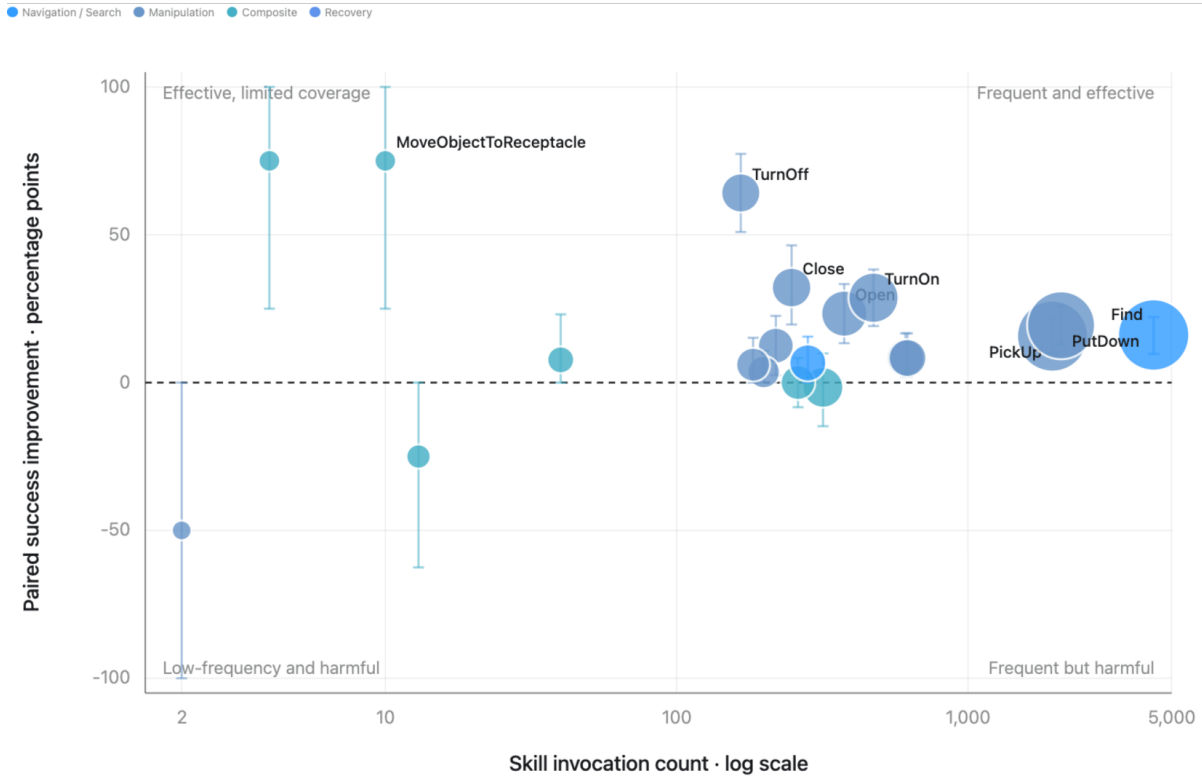


Figure 8: Skill-level benefit-risk analysis. Invocation frequency is plotted against paired success-rate improvement over the no-skill baseline; bubble size indicates task coverage, color denotes skill family, and error bars show 95% bootstrap confidence intervals.

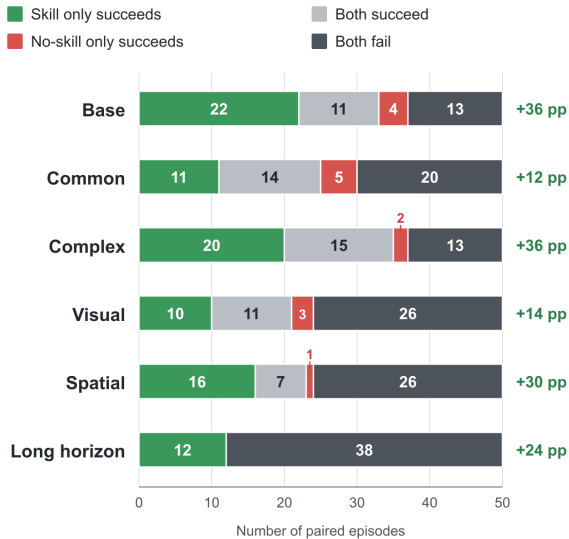


Figure 9: Paired episode outcomes between the no-skill baseline and PRACTICE across six EB-ALFRED splits. Each bar shows the four paired outcome categories, with the corresponding change in success rate (Δ SR) reported on the right.

Table 7: Skill execution and recovery statistics on EB-ALFRED. Inv.: skill invoked; Exp.: valid expansion; Acc.: primitive-action acceptance; Fail: episodes with rejected actions; Rec.: locally recovered episodes; Succ.: task success. Each split contains 50 episodes.

Split	Inv.	Exp.	Acc.	Fail	Rec.	Succ.
Base	50/50	47/50	82.6%	31/50	30/50	66.0%
CS	49/50	42/50	81.2%	36/50	32/50	50.0%
CI	49/50	48/50	85.8%	31/50	29/50	70.0%
VA	49/50	41/50	74.6%	40/50	34/50	42.0%
SR	50/50	44/50	76.5%	40/50	37/50	46.0%
LH	50/50	48/50	80.5%	45/50	45/50	24.0%
All	297/300	270/300	80.2%	223/300	207/300	49.7%

Table 7 analyzes skill invocation, action expansion, execution validity, and failure recovery across different splits on EB-ALFRED. The best results occur on Complex Instruction and Base, which achieve the highest task success rates (70.0% and 66.0%) together with reliable skill expansion and action acceptance. This suggests that reusable skills effectively convert diverse language instructions into stable action structures. Common Sense remains moderately successful, while Visual Appearance and Spatial Relationship

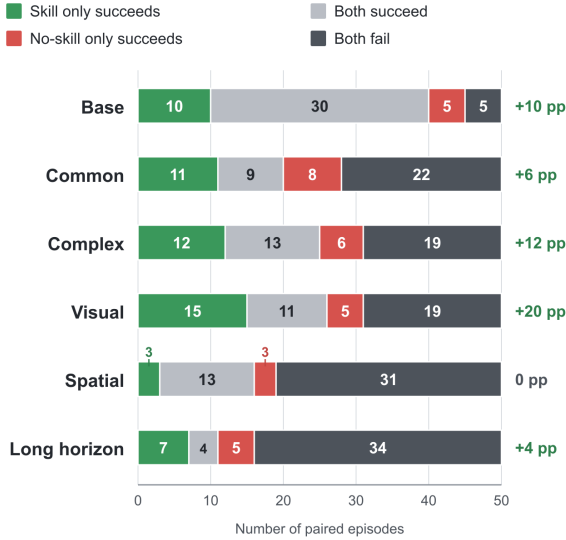


Figure 10: Paired episode outcomes between the no-skill baseline and PRACTICE across six EB-Habitat splits. Each bar shows the four paired outcome categories, with the corresponding change in success rate (Δ SR) reported on the right.

are more difficult because correct execution depends heavily on visual instance identification and spatial grounding. Long Horizon exhibits high expansion validity and recovers from every local failure, yet reaches only 24.0% task success. This indicates that local recovery is effective, but cannot fully prevent error accumulation or maintain state consistency over extended skill compositions. Overall, the results highlight the method’s strong execution and recovery capabilities while identifying visual grounding and long-horizon coordination as the main remaining bottlenecks.

D Qualitative Analysis

We qualitatively examine representative successful and failed trajectories to understand how the learned skill library supports execution and where its limitations arise. We focus on whether reusable skills reduce planning complexity, how the executor responds to rejected actions, and whether failures originate from the skill abstraction or from online grounding and execution.

D.1 Case Study in EB-ALFRED

Successful cases: light-assisted inspection. Light-assisted inspection requires the agent to coordinate object localization, illumination, and manipulation. Figure 11 shows two representative examples. In Common Sense-49, the agent grounds the indirect expressions “keys” and “one fixture on the table” to KeyChain and DeskLamp. It first invokes TURNON to establish the required lighting condition and then uses FIND and PICKUP to inspect the keys. After one rejected pickup, the executor re-localizes the keychain and completes the task without restarting the

plan. In Complex-30, the composite skill PLACEOBJECTONNONCONTAINERSURFACE grounds the remote control and armchair as a reusable placement subtask, while the executor additionally activates the floor lamp. Together, these examples demonstrate that the method can combine explicit composite skills with feedback-aware primitive execution to efficiently solve semantically diverse inspection tasks.

Successful cases: observable long-horizon transformations. Figure 12 presents two long-horizon tasks involving object transformation and relocation. Long-27 retrieves reusable FINDOBJECT cards for the knife, lettuce, and refrigerator, and composes them with slicing, cooling, and final placement actions. Long-45 similarly uses object-location skills as anchors for slicing a tomato, heating it in the microwave, and placing it in the sink. These trajectories do not rely on a single monolithic task program. Instead, the executor composes reusable skills with state-changing primitive actions and continues after several rejected actions. This modular structure allows it to preserve completed subgoals and recover locally rather than restarting the entire sequence.

Failure cases: containment and holding state. Tasks requiring one object to be placed inside another are particularly sensitive to holding-state consistency. In Base-07, the agent must insert a butter knife into a cup and then move the cup to the sink. It alternates between the two objects, repeatedly attempts pickup while holding another object, and loses the required containment relation. Complex-35 retrieves PLACEOBJECTONNONCONTAINERSURFACE for placing an apple in a pan. This skill is insufficient because the task additionally requires transporting the filled pan into the refrigerator. The executor consequently alternates between the apple and pan without establishing a stable nested state. These cases expose both incomplete skill coverage for nested transport and insufficient executor-side tracking of the object in hand.

Overall, the learned skills compress recurring multi-step behaviors into stable execution structures and enable recovery from isolated local errors. The remaining limitations largely lie outside basic skill expansion: dynamic grounding, persistent object identity, holding-state management, and long-term consistency across multiple skill calls.

D.2 Case Study in EB-Habitat

Successful cases: feedback-guided search. Unlike EB-ALFRED, EB-Habitat permits navigation only to receptacles. Object retrieval therefore requires the agent to infer a source receptacle before executing a pickup. Figure 14 shows how SEARCHANDRETRIEVE structures this process. In Base-36, the agent first searches the sofa for a toy airplane. After pickup fails, it moves to another candidate receptacle, successfully retrieves the airplane from a table, and delivers it to the left counter. Spatial-35 exhibits a longer search for a wrench: the executor visits several cabinets and counters, but preserves the same object and destination arguments until pickup succeeds. The composite skill provides a stable navigation–pickup–navigation–placement skeleton, while feedback is used to revise the uncertain source location.

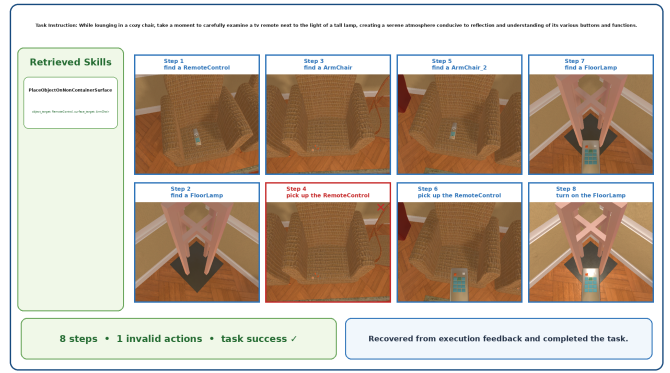
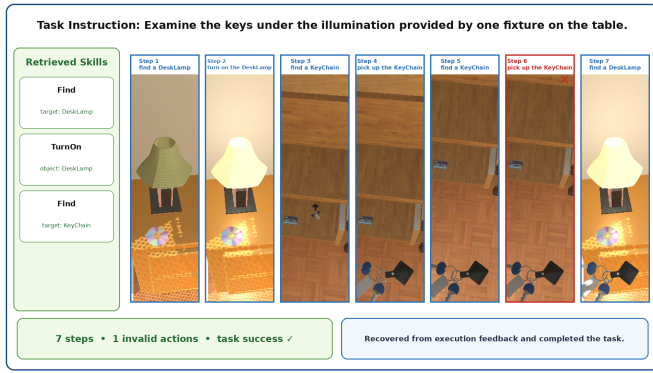


Figure 11: Successful light-assisted inspection in EB-ALFRED. Common Sense-49 (left) establishes illumination and recovers from a rejected keychain pickup, while Complex-30 (right) combines reusable object placement with lamp activation.



Figure 12: Successful long-horizon transformations in EB-ALFRED. The agent composes reusable object-location skills with slicing, cooling or heating, and final placement.



Figure 13: Containment and holding-state failures in EB-ALFRED. The executor fails to preserve the required cup–knife (left) and pan–apple (right) relations.

Successful cases: repeated multi-object transport. The same abstraction also supports tasks containing several independent relocations. In Figure 15, Long-14 sequentially moves a cleanser, sponge, and screwdriver to their respective destinations. Long-36 searches several receptacles and transports all orange instances to the brown table. Rather than generating a separate plan template for every object, the policy repeatedly instantiates SEARCHANDRETRIEVE with different

object, source, and destination arguments. This reuse reduces planning complexity while still allowing each failed pickup to trigger local search.

Failure cases: incorrect source-receptacle hypotheses. The same factor that makes SEARCHANDRETRIEVE useful also creates a Habitat-specific failure mode. In Figure 16, Base-01 begins with an incorrect source hypothesis for the pear. Although the executor visits several receptacles, it re-

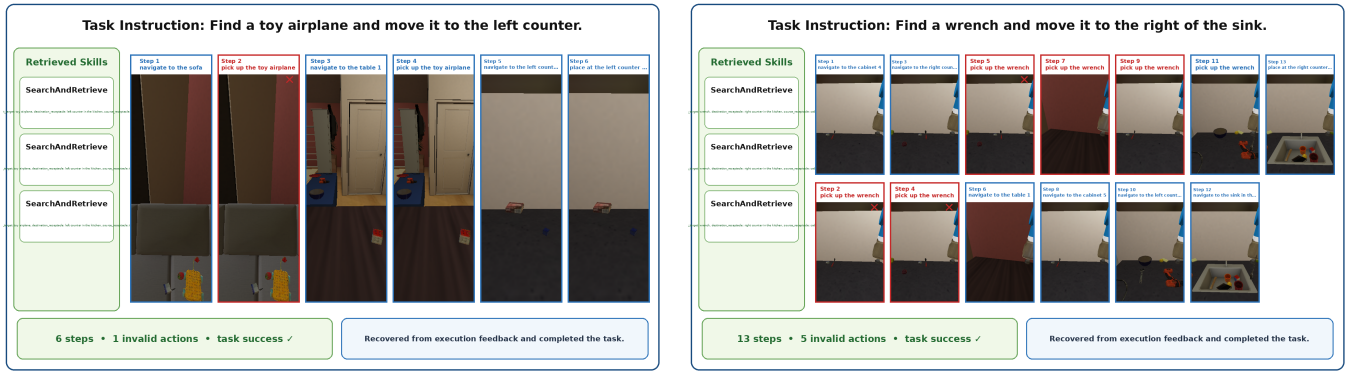


Figure 14: Feedback-guided object search in EB-Habitat. SEARCHANDRETRIEVE maintains the task structure while the executor switches candidate source receptacles.

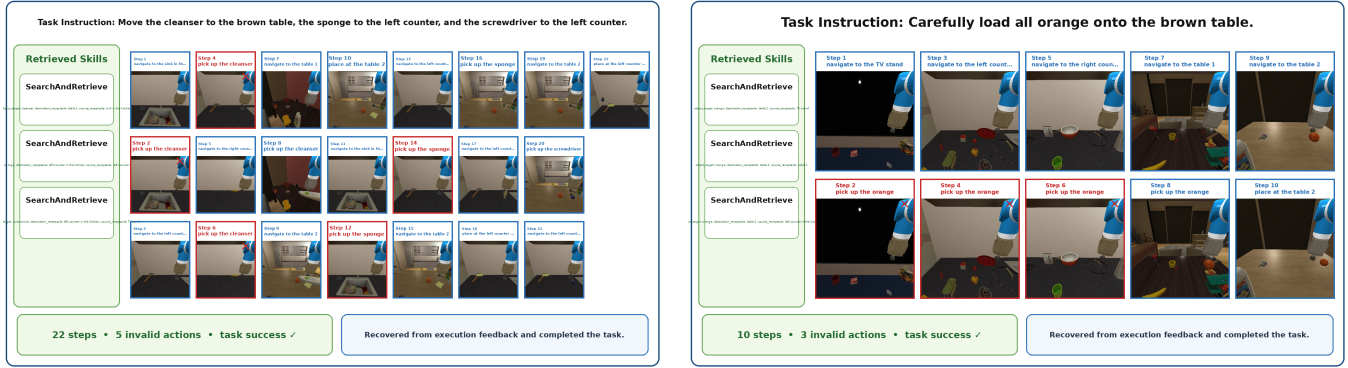


Figure 15: Successful multi-object transport in EB-Habitat. A shared SEARCHANDRETRIEVE structure is repeatedly instantiated for different objects and destinations.

peatedly attempts pickup at locations where the object is not reachable and eventually explores unrelated containers. Base-26 similarly searches tables, counters, cabinets, and drawers for an orange, but lacks a coverage-aware strategy for deciding which receptacle to inspect next. The skill schema correctly represents object transport, but its source argument is uncertain; the eventual failure is primarily caused by executor-side exploration and recovery rather than by the navigation–pickup–placement pattern itself.

Shared and environment-specific observations. Both environments exhibit failures from semantic grounding, repeated invalid-action loops, multi-object identity tracking, and error accumulation over long trajectories. Their dominant bottlenecks, however, differ. EB-Habitat restricts navigation to receptacles, making object retrieval dependent on source-receptacle inference, indirect exploration, and the mapping of expressions such as “right of the sink” to simulator receptacles. Its long-horizon tasks mainly accumulate search costs across several relocations. EB-ALFRED permits more direct object search but places greater pressure on manipulation state, including holding constraints, object containment, and transformations such as slicing, washing, heating, and cooling.

Overall, skills in EB-Habitat are most valuable for structur-

ing repeated object search and transport, whereas their main limitation is not the transport pattern itself but uncertain source-receptacle inference and ineffective exploration after failure. In contrast, EB-ALFRED is more strongly limited by nested manipulation, holding-state consistency, and long-horizon object-state transitions. Across both benchmarks, the learned library provides reusable planning structure, while robust grounding and state-aware execution remain essential for realizing its full benefit.

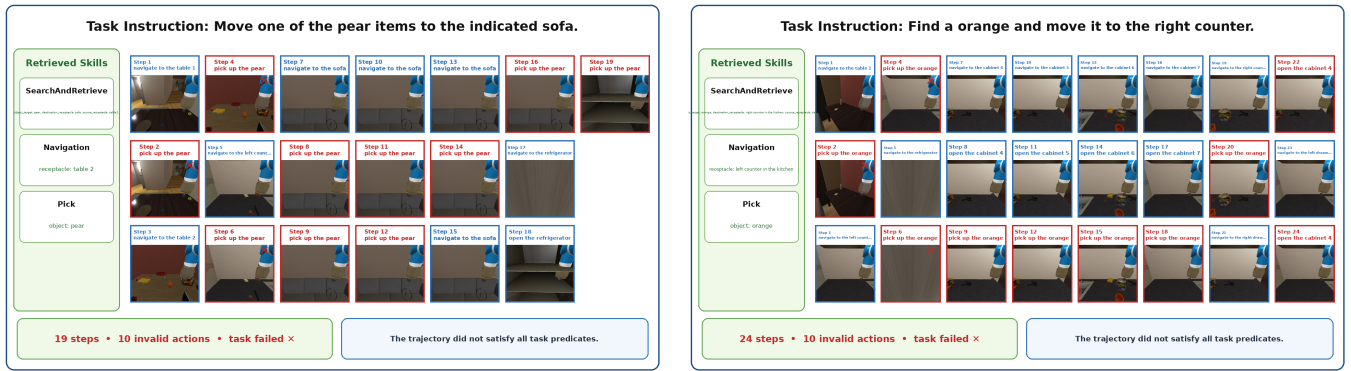


Figure 16: Source-receptacle failures in EB-Habitat. Incorrect initial hypotheses lead to repeated pickup attempts and unstructured exploration.